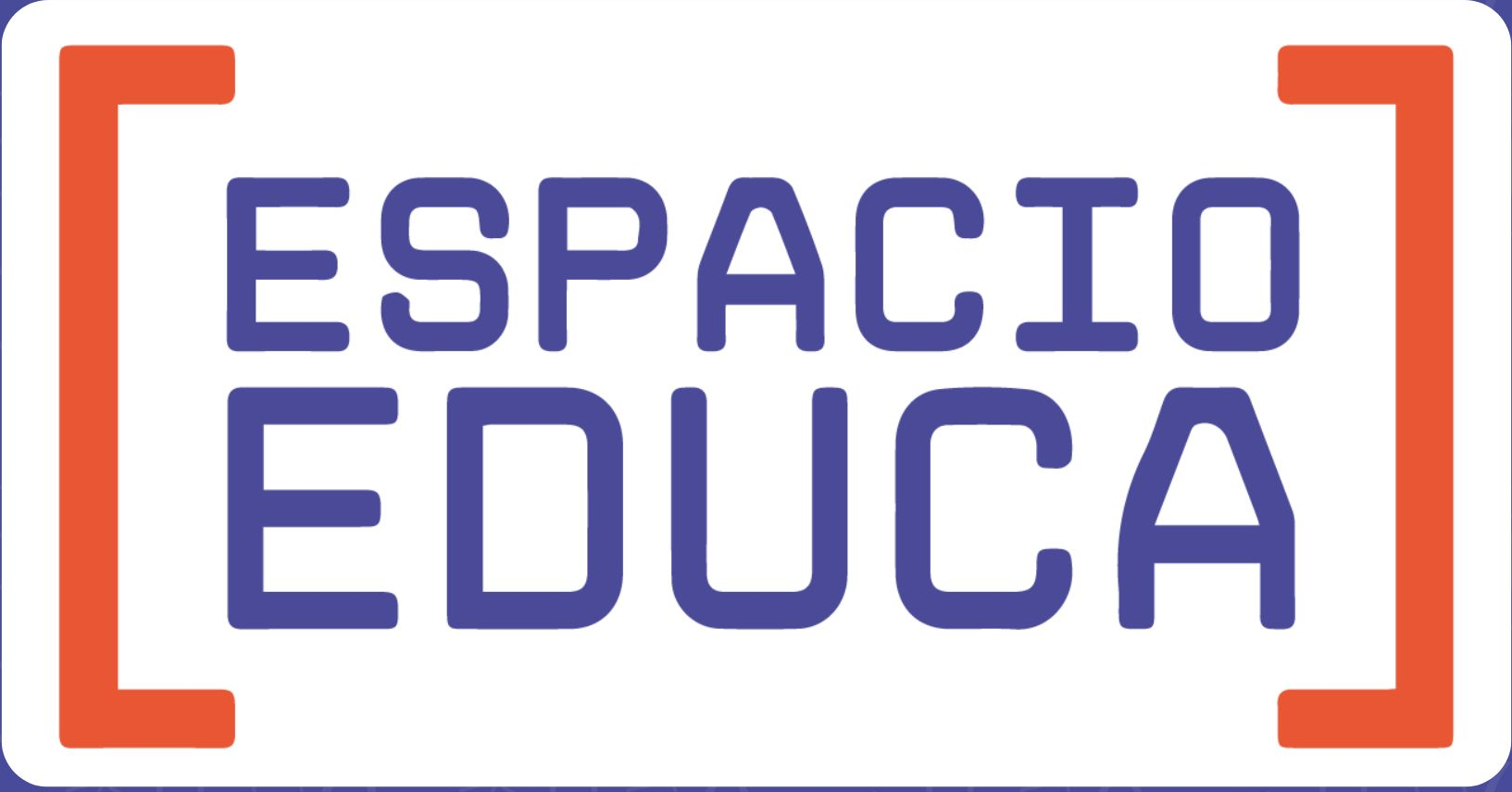




**ESPACIO  
EDUCA**

The background is a solid blue color with a repeating pattern of white icons and code symbols. The icons include computer monitors, curly braces {}, square brackets [], and less-than/greater-than signs </>. The text is centered and enclosed in large white square brackets.

# **ANUNCIOS** **ESPACIO EDUCA**

# [ HOY VAMOS A APRENDER A . . ]

CONTROL DE VERSIONES - QUE ES GIT, Y PARA QUÉ SIRVE

Vocabulario: github, colaborativo, commit



Alguna vez  
te ha pasado  
algo así?



**Cuando no sabes o no te gusta  
usar un sistema de control de  
versiones**

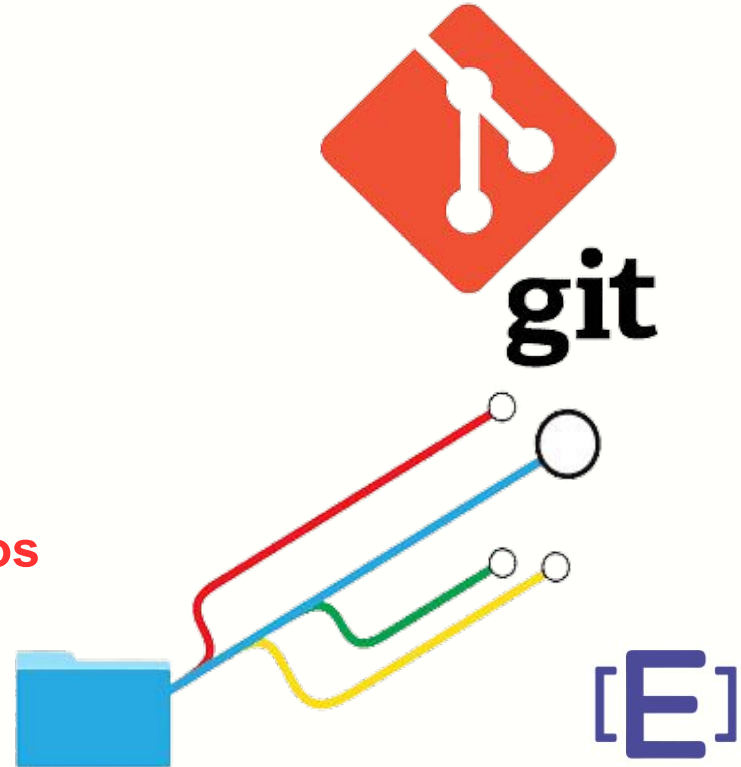
- ▶ version beta
- ▶ version final
- ▶ version final (no es la final)
- ▶ version final (no mostrar al cliente)
- ▶ version final 2.0
- ▶ version final arreglos
- ▶ version final B
- ▶ version final final final esta sí es





# ¿Qué es Git?

- Controlador de versiones **local**
- Se pueden trabajar proyectos grandes y pequeños
- Permite rastrear y controlar cambios en el código
- Velocidad y eficiencia
- Permite conectarse con **repositorios remotos**
- Se usa por línea de comando (**CLI**)
- Almacena los repositorios en el local

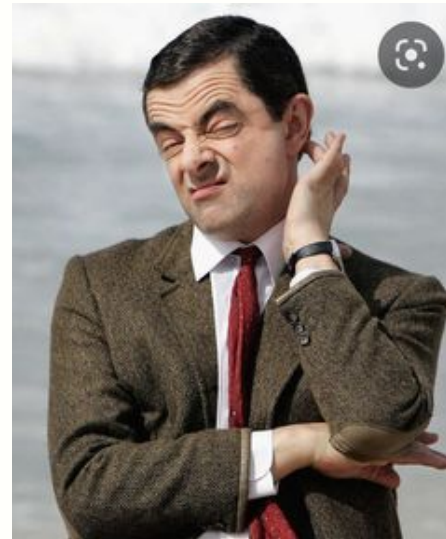
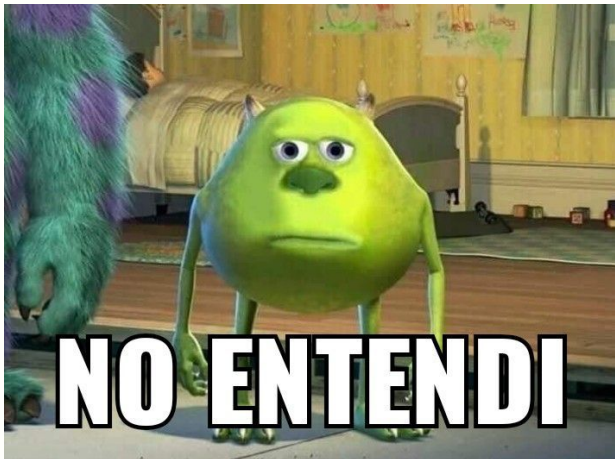




# ¿Qué es Git?

Okey, es un **controlador de versiones**, que permite **agilizar el trabajo** en proyectos grandes o pequeños y me permite llevar un **seguimiento de los cambios**.

Pero, cómo es eso?





# Conceptos básicos de git

- **Proyecto de Git:** Es como un álbum digital de tu código, donde **guardas** todas las versiones de tu programa. **Puedes compartirlo** con otros, volver atrás en el tiempo y ver cómo ha evolucionado tu proyecto.
- **Rama de Git:** Imagina que quieres experimentar con una nueva idea sin afectar la versión principal de tu programa. **Una rama es como una copia de trabajo** donde puedes hacer cambios sin miedo a estropear el original.
- **Commit de Git:** Es como guardar un **punto de control en tu proyecto**. Cada vez que haces un cambio importante, realizas un commit para registrar ese momento. Así, puedes volver a esa versión si es necesario.





## Ejemplo de git

- **Proyecto:** Un sitio web.
- **Rama principal:** La versión del sitio web que está publicada en internet.
- **Rama nueva:** Creas una rama llamada "nueva-funcionalidad" para agregar un formulario de contacto.
- **Commits:** A medida que trabajas en el formulario, haces varios commits para guardar los cambios.
- **Fusión:** Una vez que el formulario está terminado y probado, lo fusionas con la rama principal para publicarlo en el sitio web.



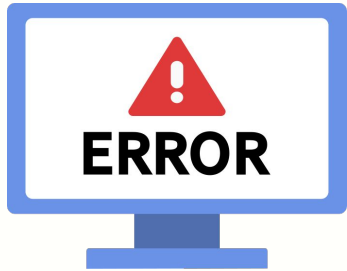


## Cómo funciona:



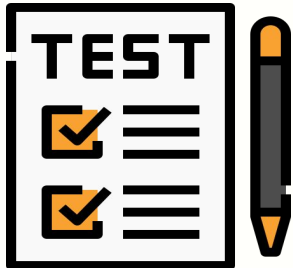


# Ventajas de usar Git



**Recuperación en caso de fallos del proyecto**

**Historial de trabajo y cambios documentados del proyecto**



**Facilidad para realizar pruebas**



# DISCUSIÓN



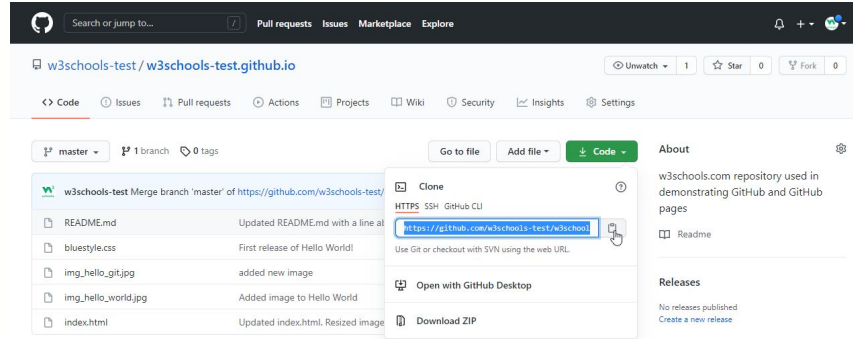
**Git clone** es el comando para descargarte el código fuente existente desde un repositorio remoto (como Github, por ejemplo). En otras palabras, Git clone básicamente realiza una copia idéntica de la última versión de un proyecto en un repositorio y la guarda en tu ordenador.

## Git Clone



```
git clone <https://link-con-nombre-del-repositorio>
```

Por ejemplo, si queremos descargar un proyecto desde Github, vamos al apartado de clonar mediante HTTP y copiamos el url y lo pegamos en el comando de arriba. Esto hará una copia del proyecto en tu espacio de trabajo local y así podrás empezar a trabajar con él.



## DISCUSIÓN



Mediante las ramas varios desarrolladores pueden trabajar en paralelo en el mismo proyecto simultáneamente. Podemos usar el comando **git branch** para crearlas, listarlas y eliminarlas.

## Git Branch

Creando una nueva rama:

```
git branch <nombre-de-la-rama>
```

Enviar (push) la nueva rama al repositorio remoto:

```
git push <nombre-remoto> <nombre-rama>
```



Pov: Clonaste un proyecto y se ve así





Para trabajar en una rama, primero tienes que cambiarte a ella. Usaremos **git checkout** principalmente para cambiarte de una rama a otra. También lo podemos usar para chequear archivos y commits.

# Git Checkout

```
git checkout <nombre-de-la-rama>
```

Hay algunos pasos que debes seguir para cambiarte exitosamente entre ramas:

1. Los cambios en tu rama actual tienen que ser confirmados o almacenados en el guardado rápido (stash) antes de que cambies de rama.
2. La rama a la que te quieras cambiar debe existir en local.
3. Hay también un comando de acceso directo que te permite crear y cambiarte a esa rama al mismo tiempo:

```
git checkout -b <nombre-de-tu-rama>
```





El comando de **git status** nos da toda la información necesaria sobre la rama actual.

# Git Status

## git status

Podemos encontrar información como:

- Si la rama actual está actualizada
- Si hay algo para confirmar, enviar o recibir (pull).
- Si hay archivos en preparación (staged), sin preparación(unstaged) o que no están recibiendo seguimiento (untracked)
- Si hay archivos creados, modificados o eliminados

```
HiMaNshU@HiMaNshU-PC MINGW64 ~/Desktop/NewDirectory (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    demofile

nothing added to commit but untracked files present (use "git add" to track)
```



## DISCUSIÓN



Cuando creamos, modificamos o eliminamos un archivo, estos cambios suceden en local y no se incluirán en el siguiente commit (a menos que cambiemos la configuración).

Necesitamos usar el comando **git add** para incluir los cambios del o de los archivos en tu siguiente commit.

## Git Add



Añadir un único archivo:

```
git add <archivo>
```

Añadir todo de una vez:

```
git add -A
```

Importante: El comando git add no cambia el repositorio y los cambios que no han sido guardados hasta que no utilicemos el comando de confirmación **git commit**.



## DISCUSIÓN



Este puede ser el comando más utilizado de Git. Una vez que queremos guardar nuestros cambios usamos **Git commit**, esto nos permite establecer un punto de control en el proceso de desarrollo al cual puedes volver más tarde si es necesario.

## Git Commit

También es importante escribir un mensaje corto para explicar qué hemos desarrollado o modificado en el código fuente.

```
git commit -m "mensaje de confirmación"
```

Importante: Git commit guarda tus cambios únicamente en local.



## DISCUSIÓN



Después de haber confirmado tus cambios, el siguiente paso que quieres dar es enviar tus cambios al servidor remoto con **Git push**.

## Git Push

```
git push <nombre-remoto> <nombre-de-tu-rama>
```

o

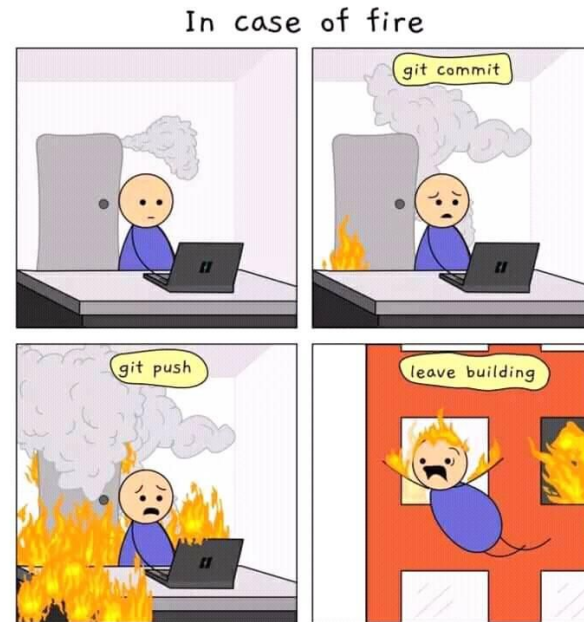
```
git push -u origin <nombre-de-tu-rama>
```

Importante: Git push solamente carga los cambios que han sido confirmados.





# Como NO hacer un commit



f /techindustan    t /techindustan    i /techindustan





El comando **git pull** se utiliza para recibir actualizaciones del repositorio remoto.

# Git Pull

```
git pull <nombre-remoto>
```

Este comando es una combinación del git fetch y del git merge lo cual significa que cuando usemos git pull recogeremos actualizaciones del repositorio remoto (git fetch) e inmediatamente aplicamos estos últimos cambios en local (git merge).

Esta operación puede generar conflictos que tengamos que resolver manualmente.



## DISCUSIÓN



Cuando ya hayas completado el desarrollo de tu proyecto en tu rama y todo funcione correctamente, el último paso es fusionar la rama con su rama padre (dev o master). Esto se hace con el comando **git merge**.

## Git Merge

Git merge básicamente integra los cambios de tu rama con todos los commits realizados a las ramas dev (o master). Es importante estar en esa rama específica que quieres fusionar con tu rama.

Por ejemplo, cuando quieres fusionar tu rama en la rama dev:

Primero, debes cambiarte a la rama dev:

```
git checkout dev
```

Antes de fusionar, debes actualizar tu rama dev local

```
git checkout fetch
```

Por último, puedes fusionar tu rama de características en la rama dev:

```
git merge <nombre-de-la-rama>
```



## DISCUSIÓN



Asegúrate de que tu rama dev tiene la última versión antes de fusionar otras ramas, **si no, te enfrentarás a conflictos u otros problemas no deseados.**

## Git Merge

GIT MERGE



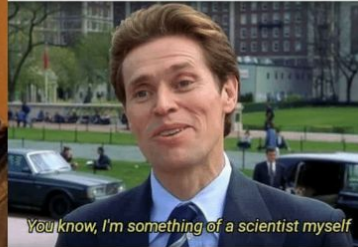
CONFLICTS

CONFLICTS EVERYWHERE

ONE DOES NOT  
SIMPLY

EXIT VIM

When you finally exit vim



after each git merge



## Types of Headaches

Migraine



Hypertension



Stress



Solving Git  
Merge conflicts



Felicidades! Eres un/a duro/a en Git!



CIERRE

